

program—so let us use that. We will create a flash binary image that includes *u-boot.bin* and the bare-metal program in it. The test program from the last article will be used here again with some modification. As the *u-boot.bin* size is around 72 KB, we will move our sample program upward in memory. In the linker script, change the starting address of the program:

```
ENTRY(_Start)
SECTIONS
{
    . = 0x100000;
    startup : { startup.o(.text)
    .data : {*(.data)}
    .bss : {*(.bss)}
    . = . + 0x500;
    sp_top = .;
}
```

Compile the test program as shown below:

```
# arm-none-eabi-gcc -c -mcpu=arm926ej-s init.c -o init.o
# arm-none-eabi-as -mcpu=arm926ej-s startup.s -o startup.o
# arm-none-eabi-ld -T linker.ld init.o startup.o -o test.elf
# arm-none-eabi-objcopy -O binary test.elf test.bin
```

Now, our test program's binary file and the (*u-boot.bin*) must be packed in a single file. Let's use the '*mkimage*' tool for this; locate it in the U-Boot source-code directory.

```
# mkimage -A arm -C none -O linux -T kernel -d test.bin -a
0x00100000 -e 0x00100000 test.uing
Image Name:
Created:    Wed Jul 6 13:29:54 2011
Image Type: ARM Linux Kernel Image (uncompressed)
Data Size: 148 Bytes = 0.14 kB = 0.00 MB
Load Address: 0x00100000
Entry Point: 0x00100000
```

Our sample binary file is ready. Let's combine it with *u-boot.bin* to create the final flash image file:

```
# cat u-boot.bin test.uing > flash.bin
```

Calculate the starting address of the test program in the *flash.bin* file:

```
# printf "%08x" $(expr $(stat -c%s u-boot.bin) + 65536)
0x21C68
```

Boot the flash image in QEMU:

```
# qemu-system-arm -M versatilepb -nographic -kernel flash.bin
```

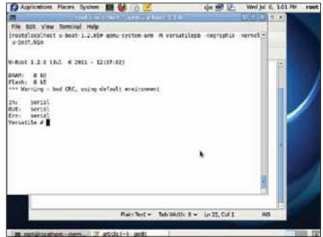


Figure 1: U-Boot

Now verify the image address in U-Boot:

```
Versatile # iminfo 0x21C68
## Checking Image at 00021c68 ...
Image Name:
Image Type: ARM Linux Kernel Image (uncompressed)
Data Size: 136 Bytes = 0.1 kB
Load Address: 00100000
Entry Point: 00100000
Verifying Checksum ... OK
```

The image is present at the address *0x21C68*. Boot it by executing the *bootm* command:

```
Versatile # bootm 0x21C68
## Booting image at 00021c68 ...
Image Name:
Image Type: ARM Linux Kernel Image (uncompressed)
Data Size: 148 Bytes = 0.1 kB
Load Address: 00100000
Entry Point: 00100000
OK
```

Starting kernel ...

Hello Open World



By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.